

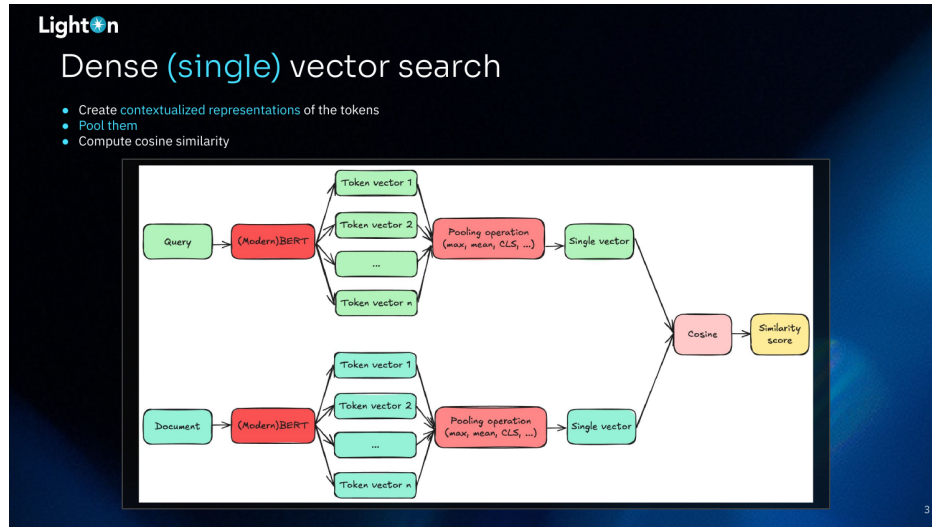
## 4 Late Interaction Models For RAG

*Based on a presentation by Antoine Chaffin*

[Antoine Chaffin](#), a researcher at LightOn, contributed to impactful open-source tools like [ModernBERT](#) and [PyLate](#), a library for working with late-interaction models.

His talk explains the intrinsic limitations of single-vector search, such as information loss from pooling, and introduces late interaction models as a more powerful alternative for modern RAG use cases like out-of-domain generalization and long context retrieval.

## 4.1 Dense Vector Search Architecture



(Timestamp: 00:03:07)

This slide diagrams the standard architecture for dense (single) vector search. A query and a document are separately fed through an encoder model (like BERT) to generate contextualized vector representations for each token. A pooling operation (e.g., max, mean, [CLS] token, etc.) then compresses all these token vectors into a single vector for the query and a single vector for the document. Finally, a similarity score (typically cosine similarity) is computed between these two vectors to determine relevance. The information loss in the pooling step is a key limitation of this approach.



88

## 4.3 The Benchmark Problem

The screenshot shows a presentation slide with a dark blue background. At the top left is the 'Lighton' logo. The title 'BEIR: a school case of Goodhart's law' is in white. Below it is a bulleted list in light blue. In the center is a white rectangular box containing the title 'BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models' and the authors' names and affiliations. A small number '6' is in the bottom right corner of the slide.

**Lighton**

### BEIR: a school case of Goodhart's law

- Big challenge of ML is generalization: models can perform well in-domain but totally fail on data they have not been trained on
- BEIR introduced various benchmarks to evaluate out-of-domain performance
- Few years and thousands of models later: the hill has been climbed
- **Run your own evaluations!**

**BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models**

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, Iryna Gurevych  
Ubiquitous Knowledge Processing Lab (UKP-TUDA)  
Department of Computer Science, Technische Universität Darmstadt  
[www.ukp.tu-darmstadt.de](http://www.ukp.tu-darmstadt.de)

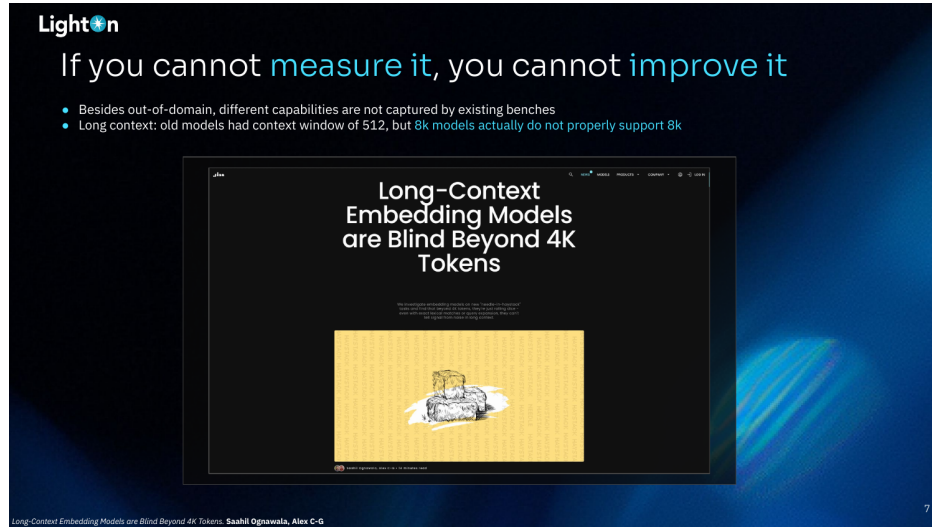
6

(Timestamp: 00:04:17)

Antoine uses the BEIR benchmark as an example of Goodhart's Law in action. BEIR was introduced to evaluate the out-of-domain generalization of retrieval models. However, as it became the standard benchmark to beat, models began to overfit to its specific datasets. Consequently, top-performing models on the BEIR leaderboard may not generalize well to new, unseen use cases, underscoring the importance of running your own evaluations on your specific data.



## 4.4 Hidden Limitations



(Timestamp: 00:05:36)

Antoine argues that if you cannot measure a capability, you cannot improve it. Existing benchmarks often miss important aspects of model performance. For instance, most older models were evaluated with a context window of only 512 tokens. While many newer models claim to support 8k tokens, recent evaluations have shown that their performance degrades significantly beyond 4k, a limitation that was not captured by older benchmarks.

## 4.5 Long Context Performance

**Lightn**

If you cannot **measure it**, you cannot **improve it**

- Besides out-of-domain, different capabilities are not captured by existing benches
- Long context: old models had context window of 512, but **8k models actually do not properly support 8k**

| Model                                    | Param. | CTX Len. | Synthetic (Acc@1) |             | Real (nDCG@10) |             |             |             | Avg.        |
|------------------------------------------|--------|----------|-------------------|-------------|----------------|-------------|-------------|-------------|-------------|
|                                          |        |          | Passkey           | Needle      | NQA            | QMS         | SFD         | WQA         |             |
| <i>512 Context Models</i>                |        |          |                   |             |                |             |             |             |             |
| E5 <sub>base</sub> (Wang et al., 2022)   | 110M   | 512      | 38.0              | 28.5        | 25.3           | 23.8        | 74.7        | 55.8        | 41.0        |
| E5-RoP <sub>base</sub>                   | 110M   | 512      | 38.5              | 31.5        | 24.6           | 23.2        | 66.6        | 38.8        | 40.9        |
| GTE <sub>base</sub> (Li et al., 2023)    | 110M   | 512      | 31.0              | 24.5        | 28.6           | 21.8        | 55.8        | 47.3        | 34.8        |
| BGE <sub>base</sub> (Xiao et al., 2023)  | 110M   | 512      | 18.0              | 25.3        | 25.6           | 22.4        | 60.3        | 51.7        | 33.9        |
| Contexter (Jacovi et al., 2021)          | 110M   | 512      | 38.5              | 29.0        | 26.7           | 25.5        | 73.5        | 47.3        | 40.1        |
| GTR <sub>base</sub> (Ni et al., 2022)    | 110M   | 512      | 38.5              | 26.3        | 26.5           | 18.3        | 63.7        | 52.2        | 36.5        |
| <i>≥ 4k Context Models</i>               |        |          |                   |             |                |             |             |             |             |
| E5-Mistral (Wang et al., 2023b)          | 7B     | 4,096    | 71.0              | 48.3        | 44.6           | 43.6        | 96.8        | 82.0        | <b>64.4</b> |
| Jina-V2 (Glincher et al., 2023)          | 137M   | 8,192    | 50.3              | 54.5        | 37.9           | 38.9        | 93.5        | 74.0        | 58.2        |
| Nomic-V1 (Nouheum et al., 2024)          | 137M   | 8,192    | 60.7              | 39.5        | 41.2           | 36.7        | 93.0        | 73.8        | 57.5        |
| BGE-M3 (Chen et al., 2024)               | 568M   | 8,192    | 59.3              | 40.5        | 45.8           | 35.5        | 94.0        | 78.0        | 58.9        |
| OpenAI-Ada-002                           | -      | -        | 50.8              | 36.8        | 41.1           | 40.0        | 91.8        | 80.1        | 56.8        |
| <i>Our Extended Models</i>               |        |          |                   |             |                |             |             |             |             |
| E5 <sub>base</sub> + Tuning (4k)         | 110M   | 4,096    | 67.3              | 41.5        | 30.4           | 35.7        | 95.2        | 69.2        | 56.6        |
| E5-RoP <sub>base</sub> + SelfExtend (4k) | 110M   | 4,096    | 72.5              | 53.5        | 32.3           | 39.1        | 91.9        | 74.6        | 60.8        |
| E5-Mistral + NTK (32k)                   | 7B     | 32,768   | <b>93.8</b>       | <b>66.8</b> | <b>49.8</b>    | <b>49.2</b> | <b>97.1</b> | <b>95.2</b> | <b>75.3</b> |

Table 2: Results (%) of existing and extended embedding models on LongEmbed. NQA, QMS, SFD, WQA is short for *NarrativeQA*, *QMSum*, *SumScreenFD*, *2WikiMultiHopQA*, respectively. We show that context window extension can effectively improve existing embedding models in processing long context.

LongEmbed: Extending Embedding Models for Long Context Retrieval. Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, Sujian Li

(Timestamp: 00:06:24)

This table from the LongEmbed paper illustrates the performance of various embedding models on long-context retrieval tasks. It shows that extending models with techniques like SelfExtend or NTK can significantly improve their ability to handle long contexts, with the E5-Mistral + NTK model achieving the highest average score.

## 4.6 Complex Retrieval Tasks

**Lightn**

If you cannot **measure it**, you cannot **improve it**

- Besides out-of-domain, different capabilities are not captured by existing benches
- Long context: old models had context window of 512, but **8k models actually do not properly support 8k**
- Retrieval beyond keyword/semantic matching is challenging

**Level 1: Keyword-based Retrieval**

Query: Who sang the song "I Think We're Alone Now"?

Relevance: Keyword matching

Positive Document: "I Think We're Alone Now" is a song ... released by the American recording artists Tony Danza and the Shondells ...

**Level 2: Semantic-based Retrieval**

Query: How *human activities* influence climate system?

Relevance: Semantic matching

Positive Document: Deforestation and urbanization result in increased emissions, urban heat island effects and changes in natural water cycle. MS MARCO, Raj et al. (2018)

**Level 3: Reasoning-based Retrieval - BRIGHT**

Query: Sustainable Living - post  
At home, after I water my plants, the water goes to plates below the pots. Can I reuse it for my plants next time?

Relevance: Risk of using recycled plant water.

Positive Document: Sustainable Living - post  
Soil *salts* are commonly found in soils. When they build up, they *destroy* the soil structure and cause direct damage to roots ...

Code - issue  
I have this table and need to transform it to ... I don't like UNPIVOT, is there a better function in snowflake for this?

Relevance: Alternative function.

Positive Document: The function `FLATTEN` flattens (explodes) compound values into multiple rows ...  
`FLATTEN` INPUT => <expr> ...

MATH - question  
Let  $k=2005^2+2^2+2008$ . What is the units digit of  $k^2+2^k$ ?

Relevance: Uses the same theorem.

Positive Document: Determine all positive integers relatively prime to all the terms of the infinite sequence  $3, 4, 5, 6, 7, 8, \dots$

LongEmbed: Extending Embedding Models for Long Context Retrieval, Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, Sujian Li

(Timestamp: 00:06:33)

Retrieval goes beyond simple keyword or semantic matching. Modern RAG systems require more complex, reasoning-based retrieval. For example, a query asking for a different Snowflake function than UNPIVOT requires understanding the function's purpose, not just matching keywords. Similarly, a math question might require retrieving a document that uses the same theorem, even if the numbers are different.

Light⬢

If you cannot **measure** it, you cannot **improve** it

- Besides out-of-domain, different capabilities are not captured by existing benches
- Long context: old models had context window of 512, but 8k models actually do not properly support 8k
- Retrieval beyond keyword/semantic matching is challenging

Table 2: The performance of retrieval models on BRIGHT. We report nDCG@10 for all datasets: Biology (Bio.), Earth Science (Earth.), Economics (Econ.), Psychology (Psy.), Robotics (Rob.), Stack Overflow (Stack.), Sustainable Living (Sust.), Text-to-Code (Text.), Poetry, ArtPS, TheoremQA with question retrieval (TheoQ) and with theorem retrieval (TheoT). Avg. denotes the average score across 10 datasets. The best score on each dataset is shown in bold and the second best is underlined. We show that reasoning-intensive retrieval is challenging for current retrievers, where the best model only achieves an nDCG@10 score of 24.3 on average. Model details are in Appendix A.1

|                                     | StackExchange |        |       |      |      | Coding |       | Theorem-based |        |       | Avg.  |       |      |
|-------------------------------------|---------------|--------|-------|------|------|--------|-------|---------------|--------|-------|-------|-------|------|
|                                     | Bio.          | Earth. | Econ. | Psy. | Rob. | Stack. | Sust. | Text.         | Poetry | ArtPS | TheoQ | TheoT |      |
| <i>Spore model</i>                  |               |        |       |      |      |        |       |               |        |       |       |       |      |
| BM25                                | 18.9          | 27.2   | 14.9  | 12.5 | 13.6 | 18.4   | 15.0  | 24.4          | 7.9    | 6.2   | 10.4  | 4.9   | 14.5 |
| <i>Open-sourced models (&lt;1B)</i> |               |        |       |      |      |        |       |               |        |       |       |       |      |
| BGE                                 | 11.7          | 24.6   | 16.6  | 17.5 | 11.7 | 10.8   | 13.3  | 26.7          | 5.7    | 6.0   | 13.0  | 6.9   | 13.7 |
| Mini-L                              | 15.2          | 21.2   | 14.7  | 22.3 | 11.4 | 13.3   | 13.3  | 19.5          | 1.3    | 8.1   | 20.9  | 9.1   | 14.2 |
| SBERT                               | 15.1          | 20.4   | 16.6  | 22.7 | 8.2  | 11.0   | 15.3  | 26.4          | 7.0    | 6.3   | 20.0  | 10.8  | 14.9 |
| <i>Open-sourced models (&gt;1B)</i> |               |        |       |      |      |        |       |               |        |       |       |       |      |
| ES                                  | 18.6          | 26.0   | 15.5  | 15.8 | 16.3 | 11.2   | 18.1  | 28.7          | 4.9    | 7.1   | 26.1  | 26.8  | 17.9 |
| SPR                                 | 19.1          | 26.7   | 17.8  | 19.0 | 16.3 | 14.4   | 19.2  | 27.4          | 2.0    | 7.4   | 24.3  | 26.0  | 18.3 |
| Mini-XL                             | 21.6          | 34.3   | 22.4  | 27.4 | 18.2 | 21.2   | 19.1  | 27.5          | 5.0    | 8.5   | 15.6  | 7.9   | 18.9 |
| GeoLM                               | 25.8          | 32.3   | 18.9  | 19.8 | 22.1 | 15.6   | 17.8  | 29.2          | 22.0   | 8.8   | 25.2  | 21.2  | 21.0 |
| Qwen                                | 30.6          | 36.4   | 17.8  | 24.6 | 13.2 | 22.2   | 14.8  | 25.5          | 9.9    | 14.4  | 27.8  | 32.9  | 22.8 |
| <i>Proprietary models</i>           |               |        |       |      |      |        |       |               |        |       |       |       |      |
| Cohere                              | 18.7          | 28.4   | 20.4  | 21.6 | 16.3 | 18.3   | 17.6  | 26.8          | 1.9    | 6.3   | 15.7  | 7.2   | 16.6 |
| OpenAI                              | 23.3          | 26.7   | 19.5  | 22.6 | 12.8 | 14.3   | 20.8  | 23.6          | 2.4    | 8.5   | 23.5  | 11.7  | 17.9 |
| Voyage                              | 23.1          | 25.4   | 19.9  | 24.9 | 10.8 | 16.8   | 15.4  | 30.6          | 1.5    | 7.5   | 27.4  | 11.6  | 17.9 |
| Google                              | 22.7          | 24.8   | 19.6  | 27.8 | 15.7 | 20.1   | 17.1  | 29.6          | 3.6    | 9.2   | 23.8  | 15.9  | 20.0 |

LongEmbed: Extending Embedding Models for Long Context Retrieval. Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, Sujian Li  
BRIGHT: A Realistic and Challenging Benchmark for Reasoning-Intensive Retrieval. Honglin Su, Howard Yen, Mengzhou Xia, Wei Ji Shi, Niklas Muenninghoff, Hao-yu Wang, Haisu Liu, Quan Shi, Zachary S. Siegel, Michael Tang, Ruosi Sun, Jinsung

(Timestamp: 00:07:27)

This table shows the performance of various retrieval models on the BRIGHT benchmark, which is designed for reasoning-intensive tasks. The results show that even large, powerful models struggle, with the best model achieving an average nDCG@10 of only 24.3.

93

## 4.7 BM25's Surprising Strength

**Light**

If you cannot **measure it**, you cannot **improve it**

- Besides out-of-domain, different capabilities are not captured by existing benches
- Long context: old models had context window of 512, but 8k models actually do not properly support 8k
- Retrieval beyond keyword/semantic matching is challenging
- BM25 (lexical search) is surprisingly strong on those challenges

| Method     | Synthetic |      | Real |      |      |      | Avg. |
|------------|-----------|------|------|------|------|------|------|
|            | P         | N    | NQA  | QMS  | SFD  | WQA  |      |
| BM25       | 100       | 95.3 | 71.5 | 81.3 | 97.6 | 96.5 | 90.4 |
| E5-Mistral | 71.0      | 48.3 | 44.6 | 43.6 | 96.8 | 82.0 | 64.4 |
| +NTK (32k) | 93.8      | 66.8 | 49.8 | 49.2 | 97.1 | 95.2 | 75.3 |

Performance of BM25 on the LongEmbed benchmark

|                            | StackExchange |        |       |      |      | Coding |      | Theorem-based |      | Avg. |      |        |        |
|----------------------------|---------------|--------|-------|------|------|--------|------|---------------|------|------|------|--------|--------|
|                            | Bin.          | Earth. | Econ. | Psy. | Rob. | Stack. | Sus. | Test.         | Pony |      | AdP  | TheoQ. | TheoT. |
| Sparse model               |               |        |       |      |      |        |      |               |      |      |      |        |        |
| BM25                       | 18.9          | 27.2   | 14.9  | 12.5 | 13.6 | 18.4   | 15.0 | 24.4          | 7.9  | 6.2  | 10.4 | 4.9    | 14.5   |
| Open-sourced models (< 8B) |               |        |       |      |      |        |      |               |      |      |      |        |        |
| BGE                        | 11.7          | 24.6   | 16.6  | 17.5 | 11.7 | 10.8   | 13.3 | 26.7          | 5.7  | 6.0  | 13.0 | 6.9    | 13.7   |
| Inte-L                     | 15.1          | 21.2   | 14.7  | 22.3 | 11.4 | 13.3   | 13.5 | 19.5          | 1.3  | 8.1  | 20.9 | 9.1    | 14.2   |
| SBERT                      | 15.4          | 20.4   | 16.6  | 22.7 | 8.2  | 11.0   | 15.3 | 26.4          | 7.0  | 5.3  | 20.0 | 10.8   | 14.9   |
| Open-sourced models (> 8B) |               |        |       |      |      |        |      |               |      |      |      |        |        |
| E5                         | 18.6          | 26.0   | 15.5  | 15.8 | 16.3 | 11.2   | 18.1 | 28.7          | 4.9  | 7.1  | 26.1 | 26.8   | 17.9   |
| SFR                        | 19.1          | 26.7   | 17.8  | 19.0 | 16.3 | 14.4   | 15.2 | 27.4          | 2.0  | 7.4  | 24.3 | 20.0   | 18.3   |
| Inte-XL                    | 21.6          | 34.3   | 22.4  | 27.4 | 18.2 | 21.2   | 15.1 | 27.5          | 5.0  | 8.5  | 15.6 | 5.9    | 18.5   |
| Gr8LM                      | 24.8          | 32.3   | 18.9  | 19.8 | 12.1 | 13.6   | 17.8 | 29.9          | 22.0 | 8.8  | 25.2 | 21.2   | 21.0   |
| Quora                      | 30.8          | 36.4   | 17.8  | 24.6 | 13.2 | 22.2   | 14.8 | 25.5          | 9.8  | 14.4 | 27.8 | 32.9   | 22.5   |
| Proprietary models         |               |        |       |      |      |        |      |               |      |      |      |        |        |
| Cohere                     | 18.7          | 28.4   | 20.1  | 21.6 | 16.3 | 18.3   | 17.6 | 26.8          | 1.9  | 6.3  | 15.7 | 7.2    | 16.6   |
| OpenAI                     | 23.1          | 26.7   | 19.5  | 22.6 | 12.8 | 14.3   | 20.8 | 23.6          | 2.4  | 8.5  | 23.5 | 11.7   | 17.9   |
| Voyage                     | 23.1          | 25.4   | 19.9  | 24.9 | 10.8 | 16.8   | 15.4 | 20.6          | 1.5  | 7.5  | 22.4 | 11.6   | 17.9   |
| Google                     | 22.7          | 34.8   | 19.6  | 27.8 | 15.7 | 20.1   | 17.1 | 29.6          | 3.6  | 9.3  | 23.8 | 15.9   | 20.0   |

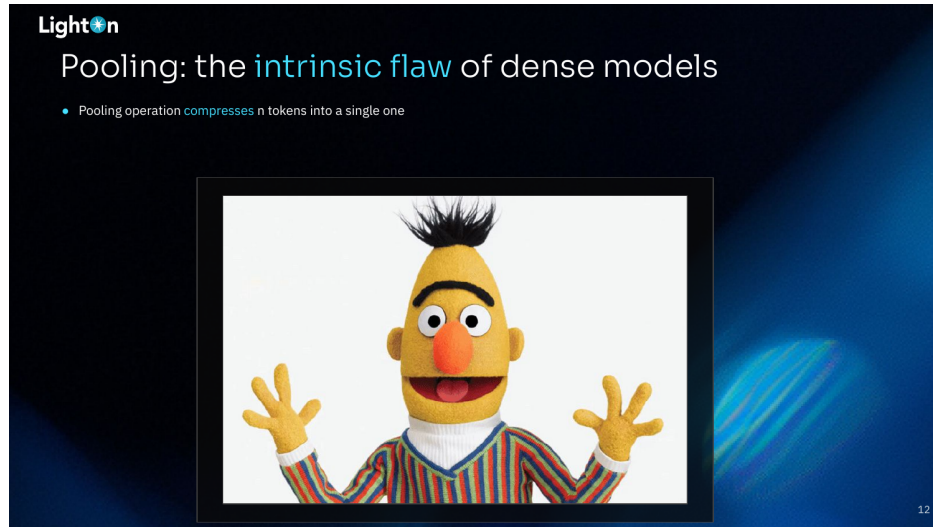
Performance of BM25 on the BRIGHT benchmark

LongEmbed: Extending Embedding Models for Long Context Retrieval. Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, Sujian Li  
BRIGHT: A Realistic and Challenging Benchmark for Reasoning-Intensive Retrieval. Honglin Su, Howard Yen, Mengzhou Xia, Wei Jin Shi, Niklas Himmelfahrt, Hao-yu Wang, Haisu Liu, Quan Shi, Zachary S. Siegel, Michael Tang, Ruoxi Sun, Jinsung

(Timestamp: 00:07:50)

Interestingly, BM25, a simple lexical search method that does not use deep learning, performs surprisingly well on these more challenging long-context and reasoning-intensive benchmarks. Its strength lies in its lack of compression; by matching exact keywords, it avoids the information loss that plagues dense models, making it a robust baseline for out-of-domain tasks.

## 4.8 The Pooling Problem



(Timestamp: 00:08:24)

Pooling is the core flaw of dense models. The process of compressing all the token vectors from a document into a single vector is inherently lossy. This compression forces the model to be selective about what information it retains.

**Lightn**

## Pooling: the **intrinsic flaw** of dense models

- Pooling operation **compresses** n tokens into a single one
- Selective behavior is learned during training (mainly through data)
- Gets more extreme with longer context
- Learn one notion of similarity

**Dense Models Learn Selective Information Encoding**

**Original Movie Review**

- 👤 **Actors:** Tom Hanks, stellar performance
- 📖 **Plot:** Complex time-travel narrative
- 🎬 **Direction:** Innovative cinematography
- 🎵 **Music:** Hans Zimmer's epic score
- 🎭 **Themes:** Love, sacrifice, redemption
- 🎨 **Visual Effects:** Groundbreaking CGI

Training with actor queries

**Dense Model**  
Learns to compress based on training data  
Single Vector Output

**Encoded Representation**

- ✓ **Actors:** Tom Hanks stellar performance
- #Plot-Complex-time-travel-narrative*
- #Direction-Innovative-cinematography*
- #Music-Hans-Zimmer's-epic-score*
- #Themes-Love-sacrifice-redemption*
- #Visual-Effects-Groundbreaking-CGI*

**Problems in Production**

**User asks about plot**

"What happens in the movie?"

✗ Information not encoded!  
Model can't retrieve relevant docs

**Cooking Recipe Input**

🍳 Ingredients: flour, eggs, butter

👨‍🍳 Chef: Gordon Ramsay

? Model confused - no actors!

**Larger Documents**

More information to compress

↓ Even more selective  
↓ More information lost

**Reasoning-Intensive Retrieval**

"Solve this mathematical problem"

Requires abstract matching

✗ Fixed similarity notion fails

13

(Timestamp: 00:08:41)

This slide illustrates how dense models learn selective information encoding. If a model is trained on a movie review dataset where queries are mostly about actors, it will learn to prioritize and encode information about actors while discarding details about the plot, music, or themes. This selective behavior leads to poor performance on out-of-domain queries (e.g., asking about the plot) or when applied to new domains entirely (e.g., cooking recipes).

**Lightn**

## Pooling: the **intrinsic flaw** of dense models

- Pooling operation **compresses** n tokens into a single one
- Selective behavior is learned during training (mainly through data)
- Gets more extreme with longer context
- Learn one notion of similarity

**BM25: Term Frequency + Inverse Document Frequency**

Query: "Leonardo DiCaprio disaster"

"Titanic features **Leonardo** **DiCaprio** in a romantic **disaster** film..."

✓ No pooling/compression  
✗ Exact matching

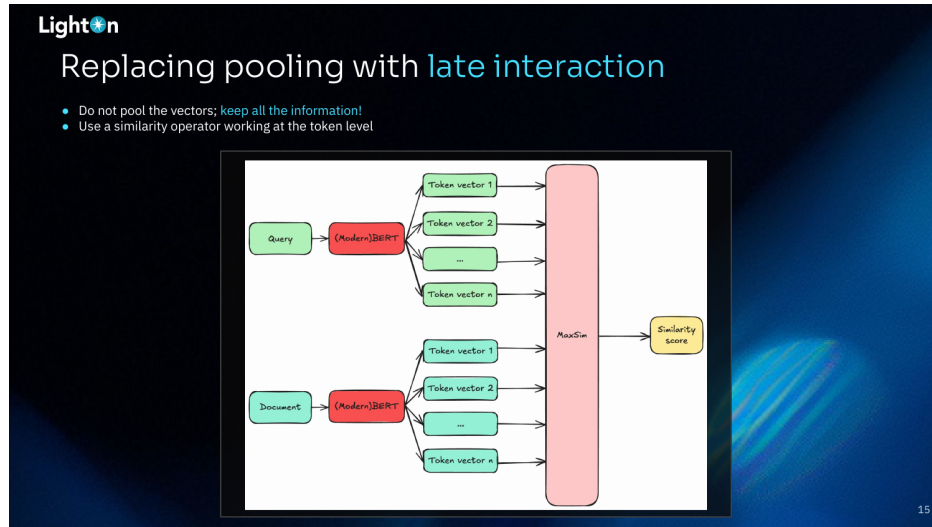
14

(Timestamp: 00:10:42)

BM25 is effective in certain cases because it avoids pooling and compression, relying on exact keyword matching. In the example, "Leonardo DiCaprio disaster" in the query directly matches the terms in the document. However, this approach fails when there's no direct lexical overlap, such as with synonyms or different languages.



## 4.9 Late Interaction Solution



(Timestamp: 00:11:32)

Late interaction models offer a solution by replacing the pooling step. Instead of compressing token vectors into a single one, they keep all the token-level information. A token-level similarity operator, such as MaxSim, is then used to compute the final score. MaxSim works by finding the maximum similarity between each query token and all document tokens, then summing these maximum scores.

**Lightn**

## This is **not** just a big single vector

- If it is a compression issue, I can just use a big embedding vector right?
- Not quite, the different "views" of a document will conflict into this single representation
- MaxSim only select one document token per query token
  - Only modify the document tokens that *matched* the query
- Can fall back to "soft lexical matching" (learned BM25)

### Dense (Single Vector)

"Titanic is a masterpiece featuring **Leonardo DiCaprio** and **Kate Winslet**. The **romantic storyline** between Jack and Rose unfolds as the **ship succumbs to disaster**. Their **forbidden love** drives the narrative."

Query: "Leonardo DiCaprio movies"    Query: "romantic disaster films"

↓ POOLING ↓

#### Single Compressed Vector

CONFLICTED REPRESENTATION

All information forced into one representation

▲ Actor and plot signals interfere with each other

### Late Interaction

"Titanic is a masterpiece featuring **Leonardo DiCaprio** and **Kate Winslet**. The **romantic storyline** between Jack and Rose unfolds as the **ship succumbs to disaster**. Their **forbidden love** drives the narrative."

Query: "Leonardo DiCaprio movies"    Query: "romantic disaster films"

↓ NO POOLING ↓

#### Token-Level Representations

|         |           |             |           |          |          |
|---------|-----------|-------------|-----------|----------|----------|
| Titanic | is        | masterpiece | Leonardo  | DiCaprio | Kate     |
| Winslet | romantic  | storyline   | between   | Jack     | and      |
| Rose    | unfolds   | as          | the       | ship     | succumbs |
| to      | disaster  | Their       | forbidden | love     | drives   |
| the     | narrative |             |           |          |          |

**MaxSim Operation:**

- Actor query matches specific actor tokens
- Plot query matches specific plot tokens
- Each aspect gets clear, uninterrupted signal

17

(Timestamp: 00:12:27)

This slide provides a clear comparison between dense and late-interaction models. A dense model forces different concepts (e.g., actors and plot) into a single, conflicted representation. In contrast, a late-interaction model maintains separate token-level representations. The MaxSim operator can then match a query about actors to the specific actor tokens and a query about the plot to the plot tokens, resulting in clean, uninterrupted signals for each aspect of the document.

## 4.10 Performance Advantages

**Lighten**

### A simple, yet effective, difference

- Strong performance out-of-domain, outperforming in-domain dense models
- Out-of-domain is tough to define, feedback from practitioners
  - Test on **your own data!**

Our model prior to the post-training step, JaColBERTv2.4, also outperforms all other existing approaches, even while being fully out-of-domain on every evaluation dataset.

similarity tasks. On 22 of 28 out-of-domain tests, ColBERTv2 achieves the highest quality, outperforming the next best retriever by up to 8% relative gain, while using its compressed representations.

of our XMOD-based dense retrievers. We observe that ColBERT-XM surpasses DPR-XM in the training language (i.e., English) by 4.5% on MRR@10. Furthermore, it consistently outperforms DPR-XM across the other 13 languages not encountered during training by an average of 4.9%. Supported by findings from Santhanam et al. (2022), our results confirm that multi-vector models bypass the restrictive information bottleneck inherent in single-vector models, enabling a richer and more nuanced representation of queries and passages, thereby yielding higher retrieval performance.

JaColBERTv2.5: Optimising Multi-Vector Retrievers to Create State-of-the-Art Japanese Retrievers with Constrained Resources. Benjamin Clavié  
ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. Keshav Santhanam, Omar Khattab, Jon Saad-Faloon, Christopher Potts, Mufid Zaharia  
ColBERTv1: A Modular Multi-Vector Representation Model for Zero-Shot Multilingual Information Retrieval. Antoine Louis, Vagish Saxena, Gjo van Dijk, Gerassimos Spanakis

18

(Timestamp: 00:13:51)

Late-interaction models like ColBERT have demonstrated strong out-of-domain performance, even outperforming in-domain dense models. Antoine emphasizes that because “out-of-domain” is hard to define, the best approach is to test these models on your own specific data to see the benefits.

**Lightn**

## A simple, yet **effective**, difference

- GTE-ModernColBERT is SOTA on the Long Embed by a large margin while being trained on document length of 300
  - Performance improves even from 8k -> 16k while even the base model has been trained at 8k max

| Model                         | Mean  |
|-------------------------------|-------|
| GTE-ModernColBERT             | 88.39 |
| voyage-multilingual-2         | 79.17 |
| snowflake-arctic-embed-l-v2,0 | 63.73 |
| gte-multilingual-base         | 62.12 |
| bge-m3                        | 58.73 |

Performance (NDCG@10) of different models on the Long Embed benchmark

19

(Timestamp: 00:15:04)

The GTE-ModernColBERT model, which uses late interaction, achieves state-of-the-art results on the LongEmbed benchmark. Notably, it outperforms other models by a large margin, even though it was trained on documents with a maximum length of only 300 tokens, while the base models it's compared against were trained with an 8k context window.

**Lighton**

## A simple, yet effective, difference

- GTE-ModernColBERT is SOTA on the Long Embed by a large margin while being trained on document length of 300
  - Performance improves even from 8k -> 16k while even the base model has been trained at 8k max
- Reason-ModernColBERT outperforms all 7B models (45 times bigger) on BRIGHT, competitive with ReasonIR-8B trained on the same data

| Model / Metric              | Full mean |
|-----------------------------|-----------|
| BRIGHT                      | 14.53     |
| < 1B DS                     |           |
| DS2                         | 13.71     |
| Inst-L                      | 14.21     |
| DS20T                       | 14.9      |
| > 1B DS                     |           |
| ES                          | 17.03     |
| SPR                         | 16.3      |
| Inst-RL                     | 16.89     |
| GEMM                        | 20.95     |
| Quen                        | 22.51     |
| Proprietary                 |           |
| Colere                      | 16.6      |
| OpenAI                      | 17.07     |
| Wang                        | 17.01     |
| Google                      | 20        |
| ReasonIR data               |           |
| ReasonIR-8B                 | 24.36     |
| Reason-ModernColBERT (150M) | 25.04     |

Performance (NDCG@10) of different models, different on the BRIGHT benchmark

20

(Timestamp: 00:15:52)

On the reasoning-intensive BRIGHT benchmark, the 150M-parameter Reason-ModernColBERT outperforms all 7B-parameter models (which are 45 times larger). It is even competitive with the proprietary ReasonIR-8B model, which was trained on the same data.



## A simple, yet effective, difference

- GTE-ModernColBERT is SOTA on the Long Embed by a large margin while being trained on document length of 300
  - Performance improves even from 8k -> 16k while even the base model has been trained at 8k max
- Reason-ModernColBERT outperforms all 7B models (45 times bigger) on BRIGHT, competitive with ReasonIR-8B trained on the same data

| Model                                 | Mean  |
|---------------------------------------|-------|
| Dense (single vector) model           | 12.31 |
| Late-interaction (multi vector model) | 19.61 |

Comparison of single and multi vector models on the BRIGHT benchmark

21

(Timestamp: 00:16:30)

This slide provides a direct, apples-to-apples comparison on the BRIGHT benchmark. A late-interaction model achieves a mean score of 19.61, while a dense (single vector) model with the same backbone and training data scores only 12.31.

## 4.11 Interpretability Benefits

The screenshot displays the ColBERT web interface. At the top, the Lighton logo is visible. The main heading is "Interpretability: a nice little bonus". Below this, two bullet points are listed: "Granular matching: understand why the document matched" and "Identify sub-chunk of the document that matches". The central part of the interface shows a "ColBERT query-passage scoring interpretability" section. It includes a "Query (max 32 tokens)" field with the text "Effects of climate change on marine ecosystems". Below this is a "Passage (max 100 tokens)" field with a longer text snippet about climate change impacts on marine ecosystems. A green button labeled "Run ColBERT scoring for query - passage" is present. Below the button, the "MaxSim Score: 27.90" and "Estimated relevance: 87.17%" are displayed. A "Contextualised Highlights" section shows the original passage with specific words highlighted in red and green to indicate relevance. The Vespa logo is at the bottom right of the interface. The background of the slide is dark blue with a glowing orb.

Lighton

### Interpretability: a nice little bonus

- Granular matching: understand why the document matched
- Identify sub-chunk of the document that matches

ColBERT

ColBERT query-passage scoring interpretability

Query (max 32 tokens)

Effects of climate change on marine ecosystems

Passage (max 100 tokens)

The changing climate has profound impacts on marine ecosystems, rising temperatures, ocean acidification, and altered precipitation patterns all contribute to shifts in the distribution and behavior of marine species, influencing the delicate balance of underwater ecosystems.

Run ColBERT scoring for query - passage

MaxSim Score: 27.90  
Estimated relevance: 87.17%

Contextualised Highlights

The changing climate has profound impacts on marine ecosystems - rising temperatures, ocean acidification, and altered precipitation patterns all contribute to shifts in the distribution and behavior of marine species - influencing the delicate balance of underwater ecosystems.

Vespa

<https://colbert.laion.ai/> Vespa

22

(Timestamp: 00:16:48)

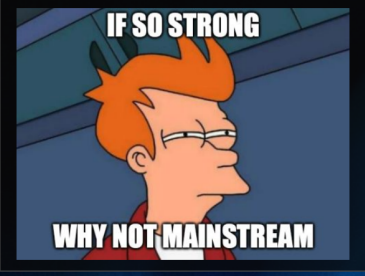
Interpretability is a valuable bonus of late-interaction models like ColBERT. Because the MaxSim operator performs granular, token-level matching, it's possible to see exactly which parts of a document contributed to the match. This allows you to identify the specific sub-chunk of text that is most relevant, which is useful for debugging and for providing more precise context to an LLM in a RAG pipeline.

## 4.12 Barriers to Adoption

Lightn

### Why people are still using dense then?

- **Storing cost:** storing  $n$  tokens instead of one
  - Adapted indexes, quantization, footprint reduction techniques
- **VectorDB support:** not originally supported by vectordb providers
  - Most of them now support it (Vespa, Weaviate, QDrant, LanceDB, Milvus, Elastic)
- **Lack of accessible tools** to train, use and experiment with these models
  - Sentence transformers is a big reason why there is are many dense models!



23

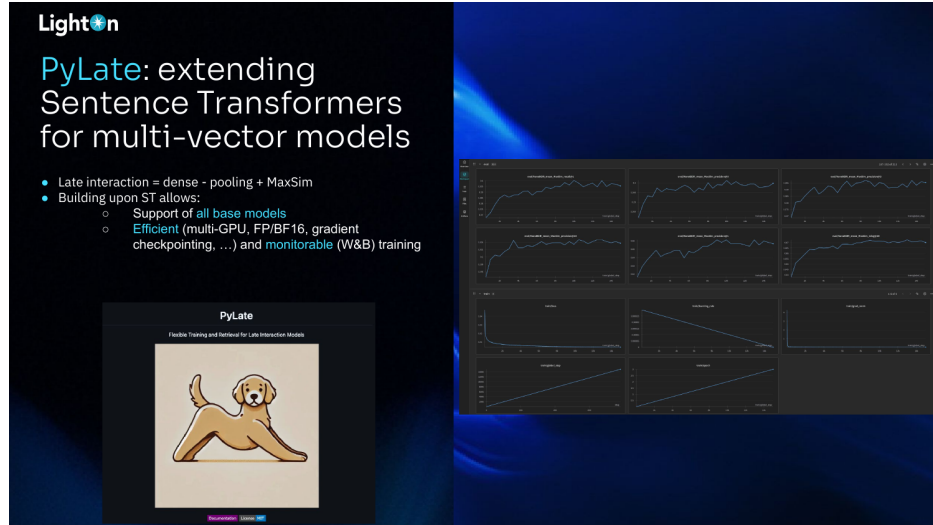
(*Timestamp: 00:17:42*)

Despite their advantages, dense models are still mainstream. Antoine attributes this to three main factors:

1. **Storing cost:** Storing  $n$  token vectors instead of one is more expensive, though techniques like quantization and footprint reduction are making this more manageable.
2. **VectorDB support:** Initially, most vector databases did not support the different search mechanism required by late-interaction models. However, this is changing, with major providers like Vespa, Weaviate, and LanceDB now offering support.
3. **Lack of accessible tools:** The widespread availability of libraries like Sentence Transformers made it very easy to work with dense models.



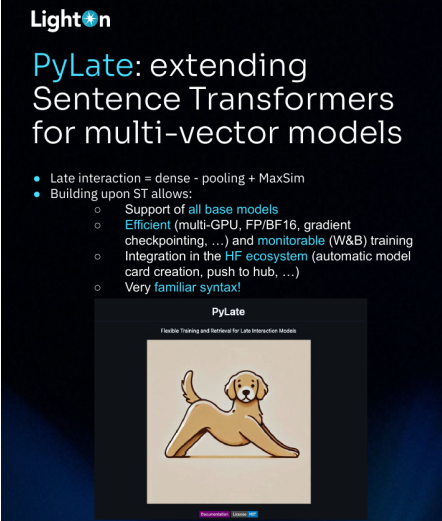
## 4.13 PyLate: Making Late Interaction Accessible



The image is a composite of two parts. The left part is a presentation slide with a dark blue background. At the top left is the 'Lightn' logo. The title 'PyLate: extending Sentence Transformers for multi-vector models' is in white and light blue. Below the title are two bullet points: 'Late interaction = dense - pooling + MaxSim' and 'Building upon ST allows:'. The second bullet point has two sub-points: 'Support of all base models' and 'Efficient (multi-GPU, FP/BF16, gradient checkpointing, ...) and monitorable (W&B) training'. At the bottom of the slide is a small image of a golden retriever standing on a white background, with the text 'PyLate' and 'Flexible Training and Inference for Late Interaction Models' above it. The right part of the image is a dashboard with a dark blue background, displaying a grid of 12 line charts showing various training metrics over time.

(*Timestamp: 00:18:43*)

To address the lack of accessible tools, Antoine and his collaborators created **PyLate**, a library that extends the popular Sentence Transformers framework for multi-vector models. Since late interaction is essentially a dense model without pooling and with a MaxSim operator, PyLate can leverage the existing Sentence Transformers ecosystem.



**PyLate**

Enabling Training and Inference for Late Interaction Models

*(Cartoon dog illustration)*

```

from datasets import load_dataset
from sentence_transformers import (
    SentenceTransformerTrainer,
    SentenceTransformerTrainingArguments,
)

from pylate import evaluator, losses, models, utils

# Define model architecture for contrastive learning
model_name = "sentence-transformers-base" # Choose the pre-trained model you want to use as base
batch_size = 32 # Larger batch size often improves results, but requires more memory

num_epochs = 10 # Adjust based on your requirements
# You can use None for "None" and model checkpointing
run_name = "sentence-transformer-base-contrast"
model_dir = "models/PyLate"

# 1. Here we define our custom model. If you a custom model, will add a custom layer to the base encoder.
model = models.CustomModel(model_name, model_dir)

# 2. Load dataset
dataset = load_dataset("sentence-transformers/sentence-embedding", "triplet", split="train")
# Split the dataset (this dataset does not have a validation set, so we split the training set)
split = dataset.train_test_split(split_test_size=0.1)
train_dataset = split["train"]
eval_dataset = split["test"]

# 3. Define the loss function
train_loss = losses.ContrastiveModelLoss()

# 4. Initialize the evaluator
my_evaluator = evaluator.SentenceTransformerEvaluator()

# 5. Configure the training arguments (e.g., batch size, evaluation storage, logging setup)
args = SentenceTransformerTrainingArguments(
    output_dir=model_dir,
    num_epochs=num_epochs,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    split_batches=True, # Set to True if you want to split your GPU across multiple GPUs
    bf16=True, # Set to True if you have a GPU that supports bf16
    run_name=run_name, # Will be used to add to wandb, if available
    logging_level=logging.INFO,
)

# 6. Initialize the trainer for the contrastive training
trainer = SentenceTransformerTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    evaluator=my_evaluator,
    data_collator=utils.CustomDataCollator(model, losses),
)

# 7. Start the training process
trainer.train()
        
```

(Timestamp: 00:20:08)

The syntax for training models with PyLate is designed to be very similar to Sentence Transformers. This familiarity makes it easy for developers to adapt their existing boilerplates and workflows. The example code shows how to define a model, load a dataset, configure training arguments, and start training with just a few modifications to a standard Sentence Transformers script.



## Training is cool, show me the evals

- Built-in efficient PLAID indexes to easily and efficiently perform retrieval
- Helper functions to evaluate the output using `ranx`
- Standard `queries/documents/qrels` format
  - MTEB
  - IR-datasets
  - BEIR
- In-training evaluator
- Integrated in FreshStack, soon in MTEB

```

1 """Evaluation script for BEIR datasets with a PLAID index."""
2 import argparse
3 from pylate import evaluation, indexes, models, retrieval
4
5 if __name__ == "__main__":
6     parser = argparse.ArgumentParser(description="Evaluates a model")
7     parser.add_argument(
8         "-i", "--index",
9         default="HfRanking",
10        help="Name of the dataset to evaluate on (default: 'HfRanking')."
11    )
12     args = parser.parse_args()
13     dataset_name = args.index_name
14     model_name = "lighton/ir-hf-ranking-v1"
15     model = models.HfRanking
16     model_name_at_path = model_name
17
18     documents, queries, qrels = evaluation.load_beir(
19         dataset_name=dataset_name, split="test"
20     )
21     index = indexes.PLAID(
22         index_name=f"lighton_{dataset_name}_{model_name}_plaid",
23     )
24     retriever = retrieval.HfRanking(index=index)
25
26     documents_embeddings = model.encode(
27         documents[dataset_name] for document in documents,
28         batch_size=1000,
29         in_memory=True,
30         show_progress_bar=True,
31     )
32
33     index.add_documents(
34         documents_ids=[document["id"] for document in documents],
35         documents_embeddings=documents_embeddings,
36     )
37     queries_embeddings = model.encode(
38         queries.values(),
39         in_memory=True,
40         show_progress_bar=True,
41         batch_size=1,
42     )
43
44     scores = retriever.retrieve_queries_embeddings_queries_embeddings, k=100)
45
46     evaluation_scores = evaluation.evaluate(
47         documents_embeddings,
48         queries_embeddings,
49         queries_ids=[query["id"] for query in queries],
50         metrics=["map", "recall@10", "recall@20", "recall@50", "recall@100"],
51     )
52     print(evaluation_scores)
            
```

(*Timestamp: 00:21:28*)

PyLate is not just for training; it also provides tools for evaluation. It includes a built-in, efficient index based on PLAID for fast retrieval. It also has helper functions that use the `ranx` library to easily compute standard IR metrics (like NDCG and Recall) on the retrieval output.

## 4.14 Future Research Directions

**Lightn**

What are the future **avenues**?

- Reduce the **cost** of multi-vector models, find the optimal trade-off

|                          | Index Size |
|--------------------------|------------|
| 16-bit Dense Vector      | 345 MB     |
| <b>2-bit PLAID Index</b> | 760 MB     |
| pool 2                   | 388 MB     |
| pool 3                   | 260 MB     |
| pool 4                   | 195 MB     |
| pool 6                   | 131 MB     |

Table 3: An overview of index size (in Megabytes) between single-vector representation and PLAID ColBERT indexes.

|                                      | cfever       | dbpedia      | fever        | figa         | hotpot       | nf           | nq           | scidocs | scifact      | covid | touche       | Avg.         |
|--------------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------|--------------|-------|--------------|--------------|
| <b>nDCG@10</b>                       |              |              |              |              |              |              |              |         |              |       |              |              |
| ColBERTv2                            | 0.163        | 0.434        | <b>0.751</b> | <b>0.338</b> | <b>0.679</b> | <b>0.329</b> | <b>0.554</b> | 0.154   | <b>0.638</b> | 0.705 | 0.261        | <b>0.455</b> |
| <b>Hierarch. Pooling<sub>2</sub></b> | 0.170        | <b>0.438</b> | 0.744        | 0.331        | <b>0.657</b> | <b>0.327</b> | <b>0.553</b> | 0.153   | 0.629        | 0.719 | 0.294        | <b>0.456</b> |
| Hierarch. Pooling <sub>3</sub>       | 0.168        | 0.414        | 0.740        | 0.311        | 0.631        | 0.310        | 0.526        | 0.152   | 0.614        | 0.730 | 0.321        | 0.447        |
| Hierarch. Pooling <sub>4</sub>       | <b>0.171</b> | 0.407        | 0.738        | 0.285        | 0.604        | 0.310        | 0.503        | 0.148   | 0.578        | 0.734 | 0.333        | 0.437        |
| <b>Hierarch. Pooling<sub>6</sub></b> | <b>0.171</b> | 0.374        | 0.731        | 0.258        | 0.544        | 0.293        | 0.468        | 0.144   | 0.574        | 0.707 | <b>0.348</b> | 0.419        |

Reducing the Footprint of Multi-Vector Retrieval with Minimal Performance Impact via Token Pooling. Benjamin Clavié, Antoine Chaffin, Griffin Adams

(Timestamp: 00:22:49)

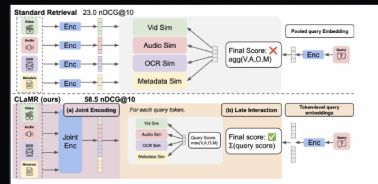
One future research avenue is to reduce the storage cost of multi-vector models. Techniques like hierarchical pooling and quantization are being explored to find the optimal trade-off between index size and performance.

Light

## What are the future avenues?

- Reduce the cost of multi-vector models, find the optimal trade-off
- Apply it to other modalities

This dual approach provides greater effectiveness with multi-vector representations, particularly in visually rich document retrieval, while maintaining efficiency for standard similarity tasks. The consistent 7-10% performance advantage of multi-vector over single-vector mode across visual tasks suggests that late interaction provides fundamentally better semantic matching for multimodal content.



ColPali: Efficient Document Retrieval with Vision-Language Models. Manuel Faysse, Hugues Sibille, Tony Wu, Bilal Omrani, Gautier Viaud, Céline Hudelot, Pierre Colombo  
 Sim-embeddings-v4: Universal Embeddings for Multimodal Multilingual Retrieval. Michael Ginter, Saba Shams, Mohammad Kalin Akran, Isabelle Helly, Andrei Ungureanu, Bo Wang, Sedigheh Estami, Scott Martens, Maximilian Werk, Nan Wang, Han Xiao  
 CLaMR: Contextualized Late-Interaction for Multimodal Content Retrieval. David Wan, Han Wang, Elias Stengel-Esklin, Jaemin Cho, Mohit Bansal

(Timestamp: 00:23:34)

Another promising direction is applying late interaction to other modalities beyond text. Approaches like ColPali have already used ColBERT for OCR-free RAG with text and images. The diagram shows the CLaMR model, which uses late interaction for multimodal content retrieval across video, audio, OCR, and metadata.



## What are the future avenues?

- Reduce the cost of multi-vector models, find the optimal trade-off
- Apply it to other modalities
- Find better similarity functions

### Efficient Document Ranking with Learnable Late Interactions

Ziwei Ji   Himanshu Jain   Andreas Veit   Sashank J. Reddi  
Sadeep Jayasumana   Ankit Singh Rawat   Aditya Krishna Menon   Felix Yu  
Sanjiv Kumar

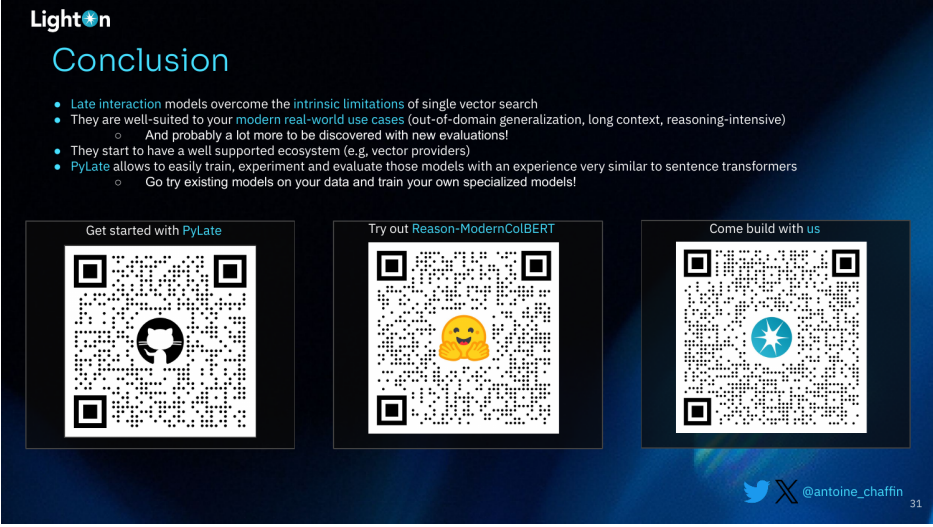
Google  
{ziwei ji, himanshu jain, andreas veit, sashank, sadeep, ankitsrawat, adityakrishnamenon, felixyu, sanjivkumar}@google.com

30

(*Timestamp: 00:24:21*)

The final future avenue is to develop better similarity functions. While the MaxSim operator is effective and has nice properties, it is relatively naive. Research into learnable late interaction functions presents an opportunity to further improve the performance of these models.

## 4.15 Chapter Summary



The slide is titled "LightOn Conclusion". It features a bulleted list of points about late interaction models. Below the list are three QR codes with icons: a GitHub icon for "Get started with PyLate", a thinking face emoji for "Try out Reason-ModernColBERT", and a star icon for "Come build with us". The slide also includes a Twitter handle "@antoine\_chaffin" and a small number "31" in the bottom right corner.

**LightOn**  
**Conclusion**

- Late interaction models overcome the **intrinsic limitations** of single vector search
- They are well-suited to your **modern real-world use cases** (out-of-domain generalization, long context, reasoning-intensive)
  - And probably a lot more to be discovered with new evaluations!
- They start to have a well supported ecosystem (e.g. vector providers)
- PyLate allows to easily train, experiment and evaluate those models with an experience very similar to sentence transformers
  - Go try existing models on your data and train your own specialized models!

Get started with PyLate

Try out Reason-ModernColBERT

Come build with us

@antoine\_chaffin 31

(Timestamp: 00:24:36)

Key takeaways from late interaction models:

1. **Overcome limitations:** Late interaction models address the fundamental information loss problem of single-vector search
2. **Modern use cases:** They excel at out-of-domain, long context, and reasoning-intensive retrieval tasks
3. **Growing ecosystem:** Tools like PyLate and vector database support make them increasingly accessible
4. **Performance gains:** Consistent improvements over dense models across challenging benchmarks
5. **Interpretability:** Token-level matching provides valuable debugging and explanation capabilities

The technology is mature enough for production use, and the performance benefits often outweigh the minor increases in storage and

computational costs.

## 4.16 Video

Here is the full video:

<https://youtu.be/1x3k0V2IITo>